

TECHNICAL DOCUMENTATION

# AeroMed DevOps Platform

Critical Care Air Ambulance Operations Platform

Case Study 75 · Cloud-Native Microservices Architecture

Highly Available · Observable · Disaster-Recovery Ready

Version 1.0

Microservices

Docker

Kubernetes (EKS)

Terraform

Jenkins CI/CD

Prometheus

Grafana

AlertManager

# Table of Contents

---

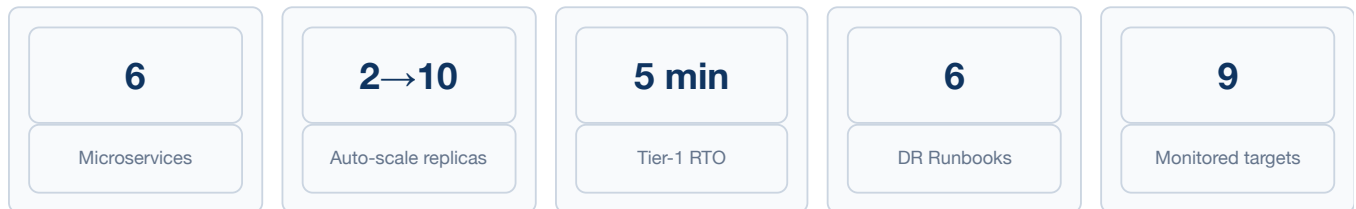
- 1 Executive Summary
- 2 Problem Statement & Objectives
- 3 System Architecture
- 4 Technology Stack
- 5 Microservices Design
- 6 Containerization (Docker)
- 7 Kubernetes Infrastructure
- 8 CI/CD Pipeline (Jenkins)
- 9 Infrastructure as Code (Terraform)
- 10 Monitoring & Observability
- 11 Security Architecture
- 12 Disaster Recovery
- 13 Deliverables Compliance Matrix
- 14 Deployment & Operations Guide
- 15 Conclusion & Future Work

# 1. Executive Summary

---

AeroMed is a production-grade DevOps platform engineered for a global air ambulance network providing critical-care transport, emergency medical evacuation, and disaster-response support.

In this domain, infrastructure failure is a **patient-safety event**: a service outage mid-evacuation can sever flight telemetry or block access to a patient's medical record during critical care. The platform is therefore architected around three non-negotiable principles — **high availability**, **observability**, and **provable disaster recovery**.



The solution delivers six independently deployable microservices behind a single API gateway, containerized with Docker, orchestrated on Kubernetes (AWS EKS), provisioned through Terraform, delivered via Jenkins CI/CD, and observed end-to-end with Prometheus, Grafana, and AlertManager. A complete disaster-recovery programme — six tested runbooks with tiered RTO/RPO objectives and scripted backup/restore/failover — completes the platform.

## 2. Problem Statement & Objectives

---

### 2.1 Business Context

AeroMed Emergency Services coordinates aircraft operations, patient records, medical-equipment monitoring, flight planning, and emergency-response workflows across multiple regions. Existing systems suffered from deployment inconsistency, limited monitoring, and poor disaster-recovery readiness.

### 2.2 Objective

Design and implement a highly available DevOps ecosystem supporting emergency healthcare operations and infrastructure resilience — capable of withstanding simulated aircraft-communication failures, medical-system outages, infrastructure disruptions, and emergency traffic surges.

### 2.3 Key Requirements

| Requirement                 | How AeroMed Addresses It                                           |
|-----------------------------|--------------------------------------------------------------------|
| Zero-downtime availability  | Multi-replica deployments, self-healing, rolling updates, multi-AZ |
| Surge tolerance             | Horizontal Pod Autoscaling (2→10 at 70% CPU)                       |
| Full observability          | Prometheus metrics, Grafana dashboards, team-routed alerting       |
| Disaster recovery           | Tiered RTO/RPO, six runbooks, scripted backup/restore/failover     |
| Reproducible infrastructure | Terraform modules for the entire AWS environment                   |
| Secure by design            | Zero-trust networking, RBAC, non-root containers, image scanning   |

### 3. System Architecture

AeroMed follows a **microservices architecture** with strict fault isolation. All external traffic enters through a single internet-facing API Gateway; the five domain services are never exposed directly. Services communicate over HTTP with explicit timeouts so that a slow or failed dependency degrades gracefully instead of cascading. Every service is stateless and independently scalable.

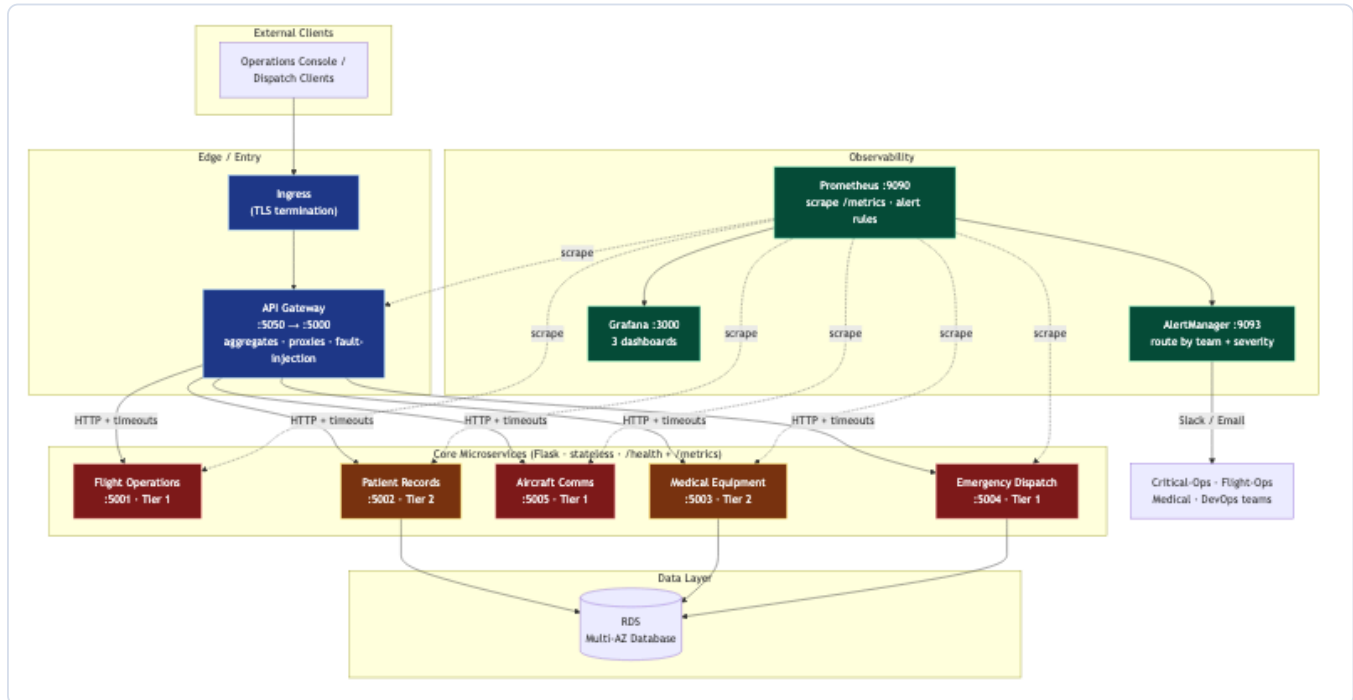


Figure 1 — Logical system architecture (Tier 1 services in red, Tier 2 in amber)

#### 3.1 Design Principles

- **Fault isolation** — a failure in one service cannot bring down another.
- **Single entry point** — only the API Gateway is public; everything else is internal-only.
- **Stateless services** — any pod can be rescheduled to any node without data loss.
- **Observability by default** — every service exposes `/health` and `/metrics`.
- **Tiered criticality** — services are graded Tier 1/2/3, which drives recovery objectives.

## 4. Technology Stack

| Layer            | Technology                            | Purpose                              |
|------------------|---------------------------------------|--------------------------------------|
| Application      | Python 3.11, Flask, Gunicorn          | Microservice runtime & WSGI server   |
| Metrics          | prometheus-client                     | Exposes service metrics              |
| Containerization | Docker                                | Immutable, portable images           |
| Orchestration    | Kubernetes (AWS EKS)                  | Scheduling, scaling, self-healing    |
| IaC              | Terraform (AWS, Kubernetes providers) | Reproducible cloud infrastructure    |
| CI/CD            | Jenkins + shared pipeline library     | Automated build, test, deploy        |
| Image security   | Trivy                                 | Vulnerability scanning in pipeline   |
| Monitoring       | Prometheus, Grafana, AlertManager     | Metrics, dashboards, alert routing   |
| Database         | AWS RDS PostgreSQL (Multi-AZ)         | Managed, highly available data store |
| Local dev        | Docker Compose                        | Single-host full-stack environment   |

## 5. Microservices Design

The platform comprises six services. Each is a Flask application exposing three standard endpoints — `/health` (liveness), `/metrics` (Prometheus), and `/api/status` (business data).

| Service            | Port      | Tier | Responsibility                                                                                                               |
|--------------------|-----------|------|------------------------------------------------------------------------------------------------------------------------------|
| api-gateway        | 5000/5050 | 3    | Single public entry point; aggregates health ( <code>/api/all-status</code> ), proxies to backends, fault-injection endpoint |
| flight-operations  | 5001      | 1    | Flight planning & mission state (origin, destination, altitude, ETA)                                                         |
| emergency-dispatch | 5004      | 1    | Emergency call handling & dispatch queue (P1/P2 priority, coordinates)                                                       |
| aircraft-comms     | 5005      | 1    | Aircraft communication, GPS telemetry, signal strength                                                                       |
| patient-records    | 5002      | 2    | Clinical records & vitals (HR, BP, SpO2, GCS)                                                                                |
| medical-equipment  | 5003      | 2    | Onboard device telemetry (ventilators, defibrillators, battery levels)                                                       |

### 5.1 Representative Endpoint (aircraft-comms)

```
@app.route("/health")          # Kubernetes liveness/readiness probe target
def health():
    return jsonify({"status": "healthy", "service": SERVICE_NAME, "version": "1.0.0"})

@app.route("/metrics")          # Prometheus scrape endpoint
def metrics():
    return generate_latest(), 200, {"Content-Type": CONTENT_TYPE_LATEST}

@app.route("/api/status")       # Business data – aircraft telemetry
def api_status():
    return jsonify({"aircraft_tracked": len(MOCK_COMMS), "communications": MOCK_COMMS})
```

**API Gateway pattern:** the gateway's `/api/all-status` fans out to all five backends and returns one aggregated health view — the single source of truth used by dashboards and the recovery demo.

## 6. Containerization (Docker)

---

Each service ships as an immutable Docker image built from a hardened, production-oriented Dockerfile.

```
FROM python:3.11-slim          # minimal base image
RUN useradd appuser            # non-root user (security)
COPY requirements.txt .        # deps first -> layer caching
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
USER appuser                   # drop root privileges
HEALTHCHECK CMD ... /health    # container-level health
CMD ["gunicorn", "--bind", "0.0.0.0:5005", "--workers", "2", "--threads", "2", "app:app"]
```

### Engineering decisions

- **Slim base image** reduces attack surface and image size.
- **Layer caching** — copying `requirements.txt` before code avoids re-installing dependencies on every code change.
- **Non-root execution** — containers never run as root.
- **Gunicorn** — a production WSGI server (multiple workers/threads), not the Flask dev server.

**Deployment evidence:** all ten containers (6 services + Prometheus + Grafana + AlertManager + Jenkins) build and run successfully via Docker Compose, exposed on ports 5050, 5001-5005, 9090, 3000, 9093, and 8080.



The diagram illustrates a CI/CD pipeline for an AWS EKS cluster. The process begins with an **Engineer** performing a **git push** to a **GitHub Repo**. This triggers a **Jenkins CI/CD** pipeline, which uses a shared library (`aeromedBuild`, `aeromedDeploy`, `aeromedNotify`) to build and push images to a **Container Registry**.

The infrastructure is provisioned by **Terraform** and consists of the following components:

- AWS (provisioned by Terraform)**:
  - VPC – networking module**:
    - Private Subnets**:
      - EKS Cluster – eks-cluster module**:
        - Monitoring workloads**: Prometheus, Grafana, AlertManager. These scrape metrics from the **Deployment: api-gateway** and **Deployments \*5 domain services**.
        - Deployment: api-gateway**: 2 replicas, HPA 2-10, @70% CPU. It receives updates via **kubectl rolling update**.
        - Deployments \*5 domain services**: each with 2 replicas, HPA 2-10, @70% CPU, liveness/readiness probes, and health checks.
        - ServiceAccounts + RBAC**: configured for least privilege.
        - Secrets and ConfigMaps**:
          - Secret: aeromed-secrets (Opaque)**: points to a **Vault migration path** and injects secrets into the domain services.
          - ConfigMap: aeromed-configmap**: injects configuration into the domain services.
        - Namespace: aeromed (RBAC + NetworkPolicy: default-deny)**: contains the domain services.
      - Public Subnets**:
        - Ingress / Load Balancer**: connects to the Internet.
    - RDS Multi-AZ**: rds module with auto failover and backups.

| Object        | Configuration & Purpose                                                                                                                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deployments   | Min 2 replicas each; <code>RollingUpdate</code> with <code>maxUnavailable: 0</code> (zero downtime); liveness/readiness probes on <code>/health</code> ; pod anti-affinity; non-root securityContext with read-only filesystem and dropped capabilities |
| Services      | Gateway = <code>LoadBalancer</code> (AWS NLB, sole public endpoint); 5 backends = <code>ClusterIP</code> (internal only)                                                                                                                                |
| ConfigMap     | Non-secret runtime config (log level, retry/circuit-breaker thresholds, 7-year HIPAA retention, service DNS)                                                                                                                                            |
| Secret        | Sensitive credentials (Opaque); documented HashiCorp Vault migration path                                                                                                                                                                               |
| HPA           | All six services scale <b>2→10</b> at <b>70% CPU</b> ; fast 30s scale-up, slow 300s scale-down to prevent flapping                                                                                                                                      |
| Ingress       | NGINX ingress controller; TLS termination; routes external traffic to the gateway                                                                                                                                                                       |
| RBAC          | Least-privilege ServiceAccounts and Roles                                                                                                                                                                                                               |
| NetworkPolicy | Zero-trust: default-deny-all-ingress, then explicit allow rules                                                                                                                                                                                         |
| Namespaces    | aeromed-production, aeromed-staging, aeromed-monitoring                                                                                                                                                                                                 |

## Self-Healing & Autoscaling Snippet

```
spec:
  minReplicas: 2          # HA floor – healthcare requires constant availability
  maxReplicas: 10         # autoscaling ceiling
  metrics:
    - type: Resource
      resource: { name: cpu, target: { averageUtilization: 70 } }
  behavior:
    scaleUp: { stabilizationWindowSeconds: 30 } # fast – emergency surges
    scaleDown: { stabilizationWindowSeconds: 300 } # slow – avoid flapping
```

## 8. CI/CD Pipeline (Jenkins)

Delivery is automated with Jenkins. A master pipeline plus one pipeline per service share a reusable pipeline library exposing three steps — `aeromedBuild`, `aeromedDeploy`, and `aeromedNotify` — so build, deploy, and notification logic live in a single versioned place.

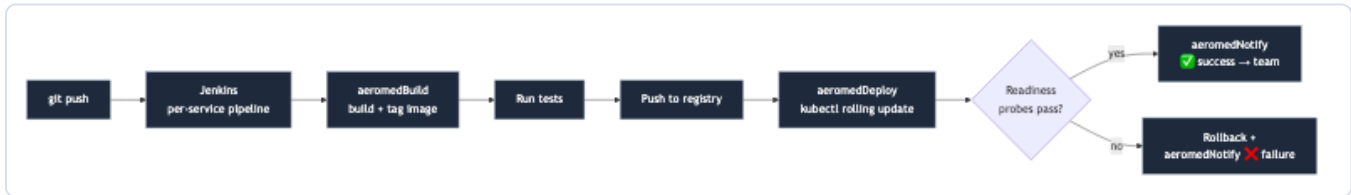


Figure 3 — CI/CD pipeline flow

### Pipeline Stages

1. **Checkout** — pull source, capture git commit.
2. **Pre-flight** — verify Docker & cluster reachability.
3. **Build** — `aeromedBuild.buildImage()` builds and dual-tags the image (build number + latest).
4. **Security Scan** — Trivy fails the pipeline on CRITICAL vulnerabilities, warns on HIGH.
5. **Unit & Integration Tests** — pytest in-container; compose-based integration with health polling.
6. **Push** — push to the container registry.
7. **Deploy** → **Staging** → **Smoke Test**.
8. **Deploy** → **Production** — gated by manual approval (rolling, zero-downtime).
9. **Verify** — post-deploy health and rollout checks; rollback on failure.

**Why a shared library?** Instead of duplicating pipeline logic across six services, hardening the deploy process once means every service inherits the improvement instantly — consistency and a single point of change.

## 9. Infrastructure as Code (Terraform)

The entire AWS environment is described in Terraform, making it reproducible and peer-reviewable. The root module composes four child modules in dependency order.

```
module "networking" { source = "../modules/networking" ... } # VPC, subnets, SGs
module "eks_cluster" { source = "../modules/eks-cluster" ...
                      depends_on = [module.networking] }      # EKS control plane + nodes
module "rds"         { source = "../modules/rds" ...
                      depends_on = [module.networking] }      # Multi-AZ PostgreSQL
module "monitoring"  { source = "../modules/monitoring" ...
                      depends_on = [module.eks_cluster, module.rds] }
```

| Module      | Provisions                                                                                         |
|-------------|----------------------------------------------------------------------------------------------------|
| networking  | VPC, public/private/database subnets, NAT gateways, security groups                                |
| eks-cluster | EKS control plane, managed node group, IAM roles, add-ons                                          |
| rds         | Multi-AZ PostgreSQL 15.4, KMS encryption with key rotation, automated backups, deletion protection |
| monitoring  | CloudWatch dashboards, alarms, SNS topics, log groups                                              |

Providers are pinned (AWS `~> 5.0` , Kubernetes `~> 2.27` ); a commented S3 + DynamoDB backend is provided for remote state with locking. A secondary DR region (eu-west-1) is defined for cross-region backups and failover.

## 10. Monitoring & Observability

The observability stack is Prometheus, Grafana, and AlertManager. Prometheus scrapes every service's `/metrics` endpoint (15s default; emergency-dispatch at 5s as it is latency-critical) and supports both static targets (Docker Compose) and Kubernetes service discovery (EKS).

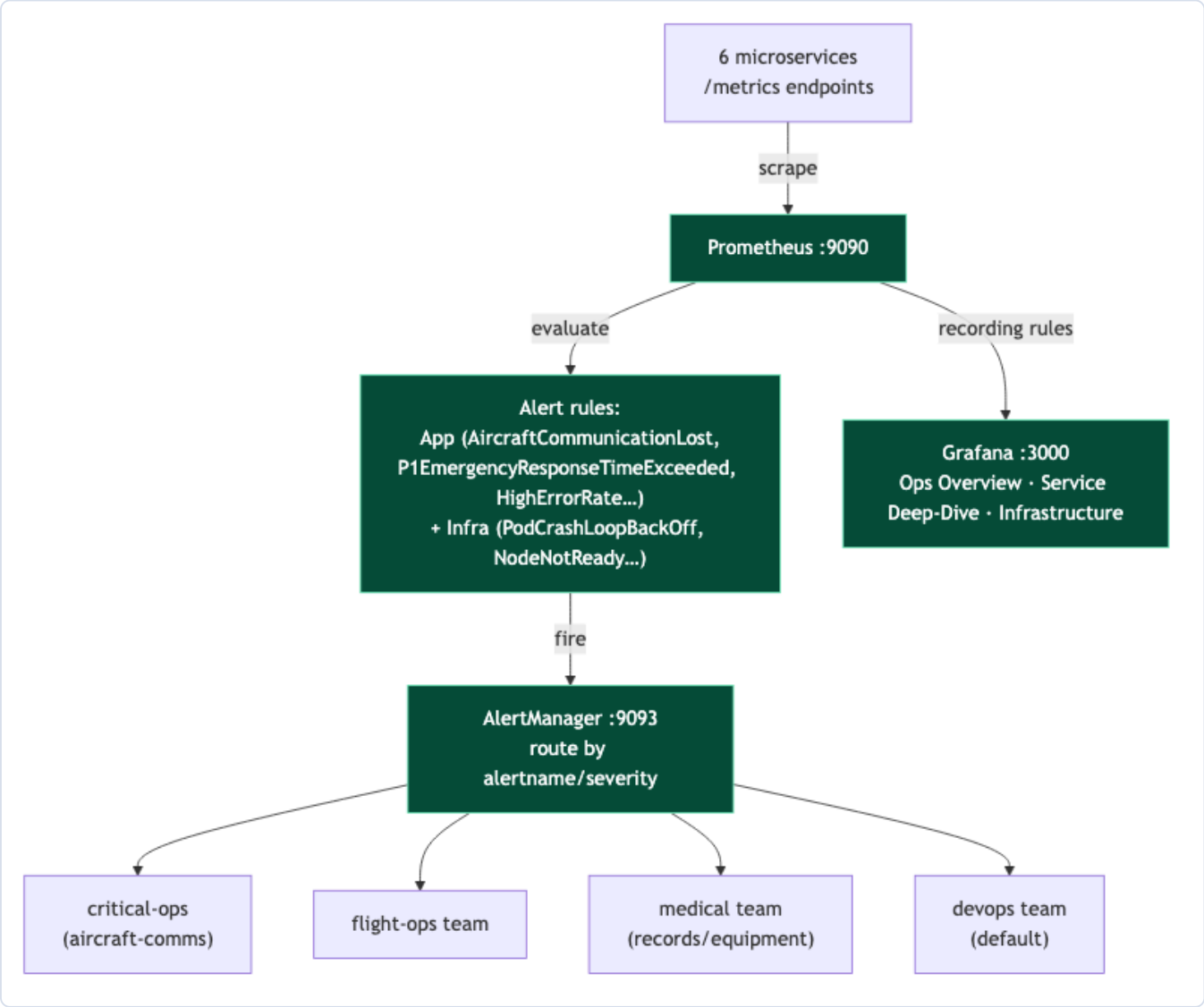


Figure 4 — Monitoring & alert-routing flow

| Component             | Role                                                                                                                              |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Prometheus            | Pull-based metric collection + alert/recording rule evaluation                                                                    |
| Application alerts    | AircraftCommunicationLost, P1EmergencyResponseTimeExceeded, HighErrorRate, EmergencyQueueBacklog                                  |
| Infrastructure alerts | PodCrashLoopBackOff, NodeNotReady, NodeMemoryPressure, HPAMaxReplicasReached                                                      |
| Grafana               | Three provisioned dashboards: Operations Overview, Service Deep-Dive, Infrastructure                                              |
| AlertManager          | Routes by alertname/severity: critical→critical-ops, aircraft→flight-ops, medical→medical-team, default→devops; via Slack + email |

**Team-aware routing** ensures the correct responder is paged for each failure class — essential when seconds influence patient outcomes.

## 11. Security Architecture

| Control              | Implementation                                                                                                                                             |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Network — Zero Trust | NetworkPolicy default-deny-all-ingress; only the gateway is externally reachable; explicit allow rules per flow                                            |
| Authorization        | RBAC with least-privilege ServiceAccounts and Roles                                                                                                        |
| Container hardening  | Non-root user, <code>readOnlyRootFilesystem</code> , <code>allowPrivilegeEscalation: false</code> , all Linux capabilities dropped, seccomp RuntimeDefault |
| Image security       | Trivy scanning in CI — CRITICAL vulnerabilities block deployment                                                                                           |
| Secrets              | Kubernetes Secrets (Opaque), isolated from ConfigMaps; documented Vault Agent Injector path                                                                |
| Data at rest         | RDS KMS encryption with automatic key rotation (multi-region key for DR)                                                                                   |
| Compliance           | HIPAA-aware: no PHI in ConfigMaps; 7-year (2555-day) records retention                                                                                     |

## 12. Disaster Recovery

---

Disaster recovery is a first-class deliverable. Recovery objectives are defined by service tier and backed by six operator runbooks plus scripted automation.



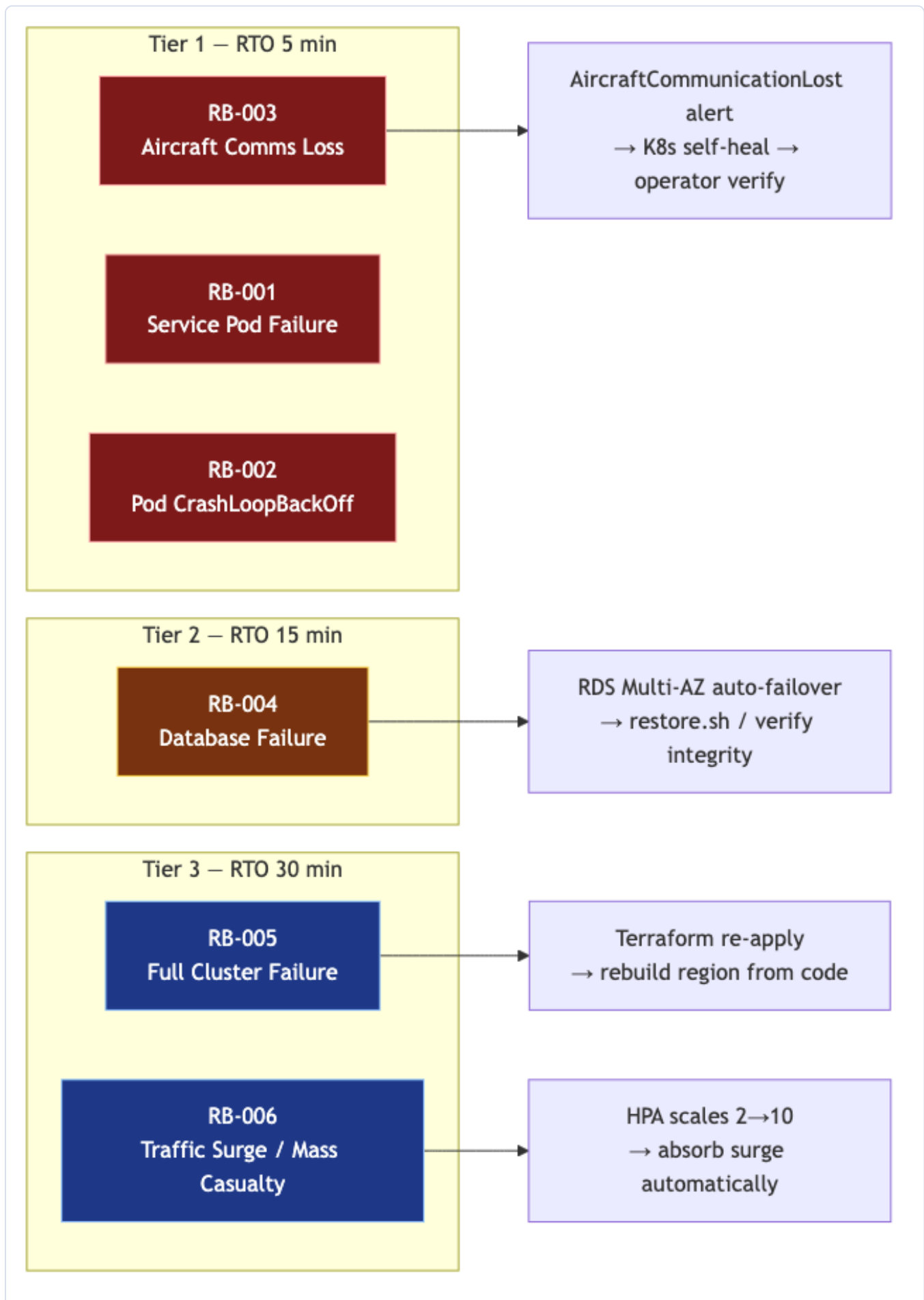


Figure 5 — Disaster-recovery map (runbooks ↔ scenarios ↔ RTO)

12.1 Recovery Objectives

| Tier   | Services                                              | RTO Target |
|--------|-------------------------------------------------------|------------|
| Tier 1 | flight-operations, emergency-dispatch, aircraft-comms | 5 minutes  |
| Tier 2 | patient-records, medical-equipment                    | 15 minutes |
| Tier 3 | api-gateway, monitoring                               | 30 minutes |

RPO for operational data is tight; Prometheus metrics tolerate up to one hour of loss (observability, not transactional data).

12.2 Runbooks

| ID     | Scenario                      | Recovery Mechanism                                   |
|--------|-------------------------------|------------------------------------------------------|
| RB-001 | Service Pod Failure           | Kubernetes self-heal; verify replica count           |
| RB-002 | Pod CrashLoopBackOff          | Diagnose logs, roll back image                       |
| RB-003 | Aircraft Communication Loss   | Alert flight-ops, restore comms within Tier-1 window |
| RB-004 | Database Failure              | RDS Multi-AZ auto-failover; integrity verification   |
| RB-005 | Full Cluster Failure          | Rebuild region from Terraform; restore from backup   |
| RB-006 | Traffic Surge / Mass Casualty | HPA scales 2→10; absorb load automatically           |

Automation scripts: `backup.sh` (exports all K8s objects + RDS snapshot to S3), `restore.sh`, `failover.sh` (DR-region switch), and `health-check-all.sh`. A `demo-full-recovery.sh` script performs a live failure-injection and recovery walkthrough.

## 13. Deliverables Compliance Matrix

| #  | Required Deliverable               | Status      | Evidence                                         |
|----|------------------------------------|-------------|--------------------------------------------------|
| 1  | Working Application                | Delivered   | 6 Flask microservices, all healthy               |
| 2  | Source Code Repository             | Delivered   | Git-managed project tree                         |
| 3  | Dockerfiles & Images               | Delivered   | 6 Dockerfiles; images build & run                |
| 4  | Jenkins CI/CD                      | Delivered   | Master + 6 pipelines + shared library            |
| 5  | Terraform IaC                      | Delivered   | 4 modules (networking, EKS, RDS, monitoring)     |
| 6  | Kubernetes Manifests               | Delivered   | Full object set across 9 categories              |
| 7  | Prometheus + Grafana               | Delivered   | 3 dashboards, alert + recording rules            |
| 8  | Disaster Recovery Plan             | Delivered   | 6 runbooks + RTO/RPO + scripts                   |
| 9  | Architecture & Deployment Diagrams | Delivered   | 5 diagrams (this document)                       |
| 10 | Project Documentation              | Delivered   | This document + docs/ guides                     |
| 11 | ELK Stack (Logging)                | Roadmap     | Structured stdout today; ELK/Loki next increment |
| 12 | Vault (Secret Management)          | Roadmap     | K8s Secrets today; documented Vault path         |
| 13 | Demonstration Screenshots          | In Progress | Capturable via demo + Grafana                    |

# 14. Deployment & Operations Guide

## 14.1 Local Environment (Docker Compose)

```
./scripts/start.sh      # build + start all 10 containers, wait for health
./scripts/status.sh    # live health table of services + infrastructure
./scripts/stop.sh      # graceful shutdown (preserves volumes)
```

| Endpoint          | URL                                        |
|-------------------|--------------------------------------------|
| API Gateway       | http://localhost:5050                      |
| Aggregated health | http://localhost:5050/api/all-status       |
| Prometheus        | http://localhost:9090                      |
| Grafana           | http://localhost:3000 (admin / aeromed123) |
| AlertManager      | http://localhost:9093                      |
| Jenkins           | http://localhost:8080                      |

## 14.2 Production (Kubernetes)

```
terraform init && terraform apply      # provision AWS infra
kubectl apply -f kubernetes/           # deploy all manifests
kubectl get pods -n aeromed-production # verify rollout
```

## 14.3 Disaster-Recovery Demo

```
./scripts/demo-full-recovery.sh      # live failure-injection + recovery walkthrough
```

## 15. Conclusion & Future Work

---

AeroMed delivers a resilient, observable, and fully automated cloud-native platform purpose-built for mission-critical healthcare transportation. Six fault-isolated microservices run on Kubernetes with autoscaling and self-healing; end-to-end monitoring provides team-aware alerting; six tested disaster-recovery runbooks tie to tiered recovery objectives; Jenkins CI/CD on a shared library guarantees consistent delivery; and Terraform makes the entire environment reproducible.

### Future Enhancements

- **Centralized logging** — integrate the ELK/Loki stack for aggregated, searchable logs.
- **Secret management** — migrate from Kubernetes Secrets to HashiCorp Vault with the Agent Injector.
- **Distributed tracing** — add OpenTelemetry for cross-service request tracing.
- **Real persistence** — replace mock data with live RDS-backed repositories.
- **Progressive delivery** — canary/blue-green deployments via Argo Rollouts.

**Document note:** Figures 1–5 are generated directly from the implemented codebase; ports, replica counts, module names, and alert names reflect the actual configuration.